

课程项目 1：用 Bernoulli Naive Bayes 做情感分类

本项目要求实现一个基于朴素贝叶斯的二分类器：输入一句英文影评，输出 0 (negative) 或 1 (positive)。需要完成词表、文本的 0/1 特征、模型训练、预测、评估和结果分析。

0. 背景

在课堂中，朴素贝叶斯处理的是由多个离散特征组成的样本，例如

$$x = (\text{天气}, \text{温度}, \text{湿度}, \text{风力})$$

并假设在给定类别 y 后，各个特征条件独立，从而有

$$P(x | y) = \prod_d P(x_d | y).$$

在本项目中，我们把同样的思想应用到文本情感分类。首先从训练集构建一个包含 D 个词的 vocabulary：

$$V = \{w_1, w_2, \dots, w_D\}.$$

对于输入的句子，取每一个词 w_d ，定义一个二值特征：

$$x_d = \begin{cases} 1, & w_d \text{ 出现在当前句子中} \\ 0, & \text{否则.} \end{cases}$$

因此，一条文本同样被表示为一个由多个离散特征组成的向量：

$$x = (x_1, x_2, \dots, x_D), \quad x_d \in \{0, 1\}.$$

之后仍然使用课堂中的朴素贝叶斯假设，并根据后验概率最大的类别完成情感预测：

$$P(x | y) = \prod_{d=1}^D P(x_d | y),$$

真实文本中的不同词之间往往存在依赖关系，因此类条件独立假设并不严格成立。本项目采用这一朴素假设，以实现并理解 Naive Bayes 分类器。

1. 数据、环境与完成标准

data/full_train.csv 是 67,349 条 SST-2 训练样本；data/full_test.csv 是 872 条有标签的 SST-2 dev 样本，本项目用它作评估集。每个 CSV 都有 text,label 两列，例如：

```
text,label
"A wonderful and beautiful film",1
"The plot was boring and awful",0
```

输入是 text 列，期望输出的类别在 label 列。只能用训练集构建词表和估计参数；评估集的文本只用于预测，标签只用于计算准确率。不得用评估集信息挑选词表或调整模型。完整数据已经提供，不需要自行下载或抽样。

在当前目录安装依赖并运行：

```
pip install -r requirements.txt
python main.py
```

只可使用 Python 标准库、numpy、pandas 实现算法，不可直接调用现成分类器（如 sklearn.naive_bayes.BernoulliNB）。src/preprocessing.py 中的 tokenize(text) 已提供，如有需求也可以自行修改实现。

2. 分词器：理解输入与输出

src/preprocessing.py 中的 tokenize(text) 为给定的分词器。输入为字符串，输出为按出现顺序排列的小写 token 列表。保留由英文字母组成的 token，以及形如 don't 的带单个撇号形式。

```
tokenize("This movie is GREAT!")
# ['this', 'movie', 'is', 'great']
```

3. 任务 A：构建词表

在 `src/preprocessing.py` 实现 `build_vocabulary(texts, max_vocab_size=5000)`。输入 `texts` 是训练文本字符串序列，`max_vocab_size` 是最多保留的 token 数。对每条文本调用给定分词器，统计每个 token 在整个训练集中的**总出现次数**。按出现次数从高到低排序，次数相同按照词汇字典序排序，保留前 `max_vocab_size` 个。输出字典 `{token: 列索引}`，索引从 0 连续编号。

```
build_vocabulary(["Good movie good", "Bad movie", "good film"], max_vocab_s
# {'good': 0, 'movie': 1, 'bad': 2}
```

4. 任务 B：生成 Bernoulli 特征

在 `src/preprocessing.py` 实现 `vectorize_texts(texts, word_to_idx)`。输入是一组文本和任务 A 的词表；输出是形状为（文本数，词表大小）的数组 X 。对于第 n 条文本和索引为 d 的词：该词至少出现一次时 $X[n, d] = 1$ ，否则为 0。重复出现仍记为 1。不在词表的 token 忽略。

```
word_to_idx = {'good': 0, 'movie': 1, 'bad': 2}
vectorize_texts(["GOOD movie good", "unknown bad"], word_to_idx).tolist()
# [[1, 1, 0], [0, 0, 1]]
```

5. 任务 C：拟合 Bernoulli Naive Bayes

在 `src/naive_bayes.py` 实现 `BernoulliNaiveBayes.fit(self, X, y)`。输入 X 是上述 0/1 特征矩阵， y 是对应的 0/1 标签序列，每一行特征对应一个标签。调用 `fit` 后，同一个模型对象应完成训练，并在 `BernoulliNaiveBayes` 中记录训练后得到的 $P(x_d | y)$ 和 $P(y = c)$ 参数。

请自行推导所需的概率计算方式，并使用 Laplace 平滑（加一平滑），避免训练数据中未出现某词时产生零概率。

6. 任务 D：预测分数和类别

在 `src/naive_bayes.py` 实现 `predict(self, X)`。`predict_probs(self, X)` 可以作为它的一个子函数去调用。输入是与训练时列顺序相同的 0/1 矩阵，可同时包含多条文本。`predict_probs` 对每条文本输出两个可比较的概率数值，分别对应标签 0 和 1。

请自行推导分数的计算方式。predict 输出与输入行数相同的标签序列，每个值为 0 或 1，选择分数较高的类别。沿用上一节训练好的 model，可以检查输出的形状、分数大小关系和最终类别：

```
X = [[1, 0], [1, 1], [0, 1], [0, 0]] # 列依次代表 good、bad
y = [1, 1, 0, 0]
model = BernoulliNaiveBayes()
model.fit(X, y)
query = [[1, 0], [0, 1]]
model.predict_probs(query).tolist()
# [[0.25, 0.75], [0.75, 0.25]]
model.predict(query).tolist()
# [1, 0]
```

注意，实现时建议在对数空间中进行中间计算，以避免大量小概率连乘造成数值下溢；在返回 predict_probs 时再将结果转换为归一化概率。这样也方便后面对倾向性样例进行数值上的比较。

7. 任务 E：输出情感词并分析

补全 main.py 末尾的 TODO。输入是训练好的模型和词表；根据训练结果，从词表中找出最倾向 positive 的 10 个词和最倾向 negative 的 10 个词（根据 $\log P(x_d = 1 \mid y = 1) - \log P(x_d = 1 \mid y = 0)$ 定义倾向），分别打印在 Most positive words 和 Most negative words 下。

例如，用词表 {'good': 0, 'bad': 1} 和下面的四条训练样本单独拟合模型，再令每组只取 1 个词：

```
X = [[1, 0], [1, 0], [0, 1], [0, 1]]
y = [1, 1, 0, 0]
model = BernoulliNaiveBayes()
model.fit(X, y)
```

此时 good 的正/负类条件概率分别是 0.75 / 0.25，bad 则相反，期望输出：

```
Most positive words:
good  1.099
Most negative words:
bad   -1.099
```

8. 提交与结果说明

提交完成后的代码，以及一份实验报告。报告说明应记录运行命令、测试准确率（当前标答在所附完整数据上输出约为80%，可用作核对）、两个方向各 10 个词及其分数，并回答：

1. 观察你得到的正向和负向词，各举一例说明它是否符合直觉。
2. 寻找几个模型预测失败的bad case，分析为什么会失败。
3. 上述实验里我们设置的vocab_size是固定的5000。尝试设置不同的vocab_size或其它超参数，比较测试集表现，并简单解释观察。
4. （选做）尝试在当前朴素贝叶斯的代码框架下提出一些改进，并通过实验验证改进方法的效果，并分析结果。（例：修改B对应的文本特征表示；修改C对应的概率估计函数；采用其他合理的 Naive Bayes 特征分布假设等）